

# Projet de Communication Inter-Microcontrôleurs

*Liaison Série Asynchrone Bidirectionnelle Virtuelle et Affichage Dynamique LCD*

**Auteur :** Cenderson

**Environnement :** Proteus ISIS & com0com

**Cible :** 2 × PIC18F2550

**Langage :** Assembleur Microchip (ASM)

## 1. Introduction et Objectifs du Projet

Ce projet s'inscrit dans le cadre des travaux d'ingénierie des systèmes embarqués du département STIC. L'objectif est de concevoir, implémenter et valider un protocole de communication asynchrone unidirectionnel entre deux microcontrôleurs **PIC18F2550**.

Le système se décompose en deux entités distinctes : un module émetteur intelligent qui intercepte les flux de données provenant d'une interface de saisie (Terminal Virtuel), assure une validation et un affichage local en temps réel, et un module récepteur autonome chargé de stocker et de faire défiler le message validé sur un second afficheur distant.

## 2. Architecture Matérielle et Affectation des Broches

Chaque microcontrôleur est interfacé avec un afficheur LCD alphanumérique 16x2 (pilote standard HD44780) configuré en **mode 4 bits** afin d'optimiser l'utilisation des lignes d'E/S du PORTB. Les modules de communication série (USART intégrés) sont configurés pour une transmission standard à **9600 bauds** (8 bits de données, pas de parité, 1 bit de stop).

### Configuration de la Liaison Série (UART)

Pour une fréquence d'oscillation de 4 MHz (oscillateur interne) et un fonctionnement en haute vitesse (BRGH = 1), le registre SPBRG est initialisé à la valeur décimale **25** pour fixer précisément le taux de transfert à 9600 bauds.

| Composant                       | Broche PIC18F2550      | Rôle / Fonctionnalité                                    |
|---------------------------------|------------------------|----------------------------------------------------------|
| Module USART                    | RC6 / TX (Pin 25)      | Ligne de transmission série des données                  |
|                                 | RC7 / RX (Pin 26)      | Ligne de réception série des données                     |
| Afficheur LCD (Local & Distant) | RB0 à RB3 (Pins 21-24) | Lignes de données de poids fort du LCD (D4 à D7)         |
|                                 | RB4 (Pin 25)           | Sélection du registre : Commande (0) / Donnée (1) - [RS] |
|                                 | RB5 (Pin 26)           | Impulsion de validation de l'écriture - [E]              |

### 3. Implémentation Physique vs Concessions de Simulation

Le passage d'un déploiement matériel à un environnement simulé sous **Proteus ISIS** nécessite de prendre en compte plusieurs abstractions et concessions techniques :

- **Liaisons et Émulation de Périphériques** : En environnement réel, la transmission initiale repose sur des modules Bluetooth (ex: HC-05) dialoguant avec un smartphone. En simulation, afin d'éliminer les couches de drivers superflues et les instabilités de synchronisation de l'hôte, le flux est injecté via le **Terminal Virtuel Proteus** connecté au RX du PIC Émetteur. De plus, la connectivité inter-PIC utilise le logiciel **com0com** associé à des blocs COMPIM pour simuler un câble Null-Modem virtuel transparent.
- **Alimentation et Reset** : Les broches d'alimentation (VDD, VSS) et de gestion du contraste des écrans (potentiomètre sur V0) sont gérées implicitement par les modèles mathématiques de Proteus. De même, la broche de réinitialisation matérielle MCLR est désactivée au niveau des fusibles de configuration (MCLRE = OFF) pour simplifier le routage logique.
- **Stabilité Horloge** : La simulation Proteus élimine la dérive thermique inhérente aux circuits RC ou aux oscillateurs internes réels. La directive CONFIG FOSC = INTOSCIO\_EC offre une base de temps parfaite à 4 MHz, garantissant l'absence d'erreurs de tramage sur l'UART.

### 4. Codes Sources Optimisés

Les programmes ci-dessous ont été fiabilisés pour éviter les conflits d'indexation de la mémoire RAM lors des défilements de chaînes de caractères.

#### CODE SOURCE 1 : PIC ÉMETTEUR INTELLIGENT

```

TITLE "Emetteur Intelligent - Sanderson"
LIST P=18F2550
#include <P18F2550.INC>

CONFIG FOSC = INTOSCIO_EC, WDT = OFF, MCLRE = OFF, LVP = OFF

CBLOCK 0x000
    temp, d1, d2, d3

```

```

        char_rx, last_char
        ptr_count
    ENDC

#define MSG_BASE 0x100

    ORG 0x0000
    GOTO start

    ORG 0x0020
start:
    MOVLW 0x60
    MOVWF OSCCON
    CLRF TRISB

    CALL reset_buffer

    CALL delay_50ms
    CALL lcd_init
    CALL init_uart

main_loop:
    BTFSS PIR1, RCIF
    BRA main_loop

    MOVF RCREG, W
    MOVWF char_rx

    XORLW '&'
    BNZ check_send
    MOVF last_char, W
    XORLW '&'
    BNZ update_last
    CALL clear_lcd_and_reset
    BRA main_loop

check_send:
    MOVF char_rx, W
    XORLW '-'
    BNZ store_and_display
    MOVF last_char, W
    XORLW '-'
    BNZ update_last
    CALL send_buffer
    BRA main_loop

store_and_display:
    MOVF char_rx, W
    CALL lcd_write_char

    MOVF ptr_count, W
    SUBLW .15
    BTFSS STATUS, C
    CALL lcd_scroll_left

    MOVF char_rx, W
    MOVWF POSTINC0

```

```

    INCF ptr_count, 1
    BRA update_last

update_last:
    MOVF char_rx, W
    MOVWF last_char
    BRA main_loop

lcd_scroll_left:
    MOVLW 0x18
    CALL lcd_cmd
    RETURN

clear_lcd_and_reset:
    MOVLW 0x01
    CALL lcd_cmd
    CALL reset_buffer
    RETURN

send_buffer:
    MOVF ptr_count, W
    BZ end_send
    LFSR FSR1, MSG_BASE
send_next:
    MOVF POSTINC1, W
    CALL send_char
    DECFSZ ptr_count, 1
    BRA send_next
    MOVLW 0x00          ; Envoi du marqueur de fin (NULL) pour le récepteur
    CALL send_char
end_send:
    CALL clear_lcd_and_reset
    RETURN

reset_buffer:
    CLRF ptr_count
    LFSR FSR0, MSG_BASE
    CLRF last_char
    RETURN

init_uart:
    BSF TRISC, 7
    BCF TRISC, 6
    MOVLW .25
    MOVWF SPBRG
    BSF RCSTA, SPEN
    BSF RCSTA, CREN
    BSF TXSTA, TXEN
    BSF TXSTA, BRGH
    RETURN

send_char:
    BTFSS PIR1, TXIF
    BRA send_char
    MOVWF TXREG
    RETURN

```

```

lcd_init:
    MOVLW 0x03
    MOVWF LATB
    CALL pulse_e
    CALL delay_5ms
    MOVLW 0x02
    MOVWF LATB
    CALL pulse_e
    MOVLW 0x28
    CALL lcd_cmd
    MOVLW 0x0C
    CALL lcd_cmd
    MOVLW 0x01
    CALL lcd_cmd
    RETURN

lcd_cmd:
    MOVWF temp
    SWAPF temp, W
    ANDLW 0x0F
    MOVWF LATB
    CALL pulse_e
    MOVF temp, W
    ANDLW 0x0F
    MOVWF LATB
    CALL pulse_e
    CALL delay_5ms
    RETURN

lcd_write_char:
    MOVWF temp
    SWAPF temp, W
    ANDLW 0x0F
    IORLW 0x10
    MOVWF LATB
    CALL pulse_e
    MOVF temp, W
    ANDLW 0x0F
    IORLW 0x10
    MOVWF LATB
    CALL pulse_e
    MOVF temp, W
    RETURN

pulse_e:
    BSF LATB, 5
    NOP
    BCF LATB, 5
    CALL delay_5ms
    RETURN

delay_5ms:
    MOVLW .203
    MOVWF d1
    MOVLW .7
    MOVWF d2

d5_L_e:

```

```

    DECFSZ d1, 1
    BRA d5_L_e
    DECFSZ d2, 1
    BRA d5_L_e
    RETURN

delay_50ms:
    MOVLW .10
    MOVWF d3
d50_L_e:
    CALL delay_5ms
    DECFSZ d3, 1
    BRA d50_L_e
    RETURN
    END

```

## CODE SOURCE 2 : PIC RÉCEPTEUR COMMUTATION FORCÉE

```

TITLE "Recepteur Commutation Forcee - Sanderson"
LIST P=18F2550
#include <P18F2550.INC>

CONFIG FOSC = INTOSCIO_EC, WDT = OFF, MCLRE = OFF, LVP = OFF

CBLOCK 0x000
    offset, char_count, temp, d1, d2, d3
ENDC

#define MSG_RAM_START 0x100

ORG 0x0000
GOTO start

delay_5ms:
    MOVLW .203
    MOVWF d1
    MOVLW .7
    MOVWF d2
d5_L:
    DECFSZ d1, 1
    BRA d5_L
    DECFSZ d2, 1
    BRA d5_L
    RETURN

delay_25ms:
    MOVLW .5
    MOVWF d3
d25_L:
    CALL delay_5ms
    DECFSZ d3, 1
    BRA d25_L
    RETURN

pulse_e:

```

```

    BSF LATB, 5
    NOP
    BCF LATB, 5
    CALL delay_5ms
    RETURN

lcd_cmd:
    MOVWF temp
    SWAPF temp, W
    ANDLW 0x0F
    MOVWF LATB
    CALL pulse_e
    MOVF temp, W
    ANDLW 0x0F
    MOVWF LATB
    CALL pulse_e
    RETURN

lcd_write_char:
    MOVWF temp
    SWAPF temp, W
    ANDLW 0x0F
    IORLW 0x10
    MOVWF LATB
    CALL pulse_e
    MOVF temp, W
    ANDLW 0x0F
    IORLW 0x10
    MOVWF LATB
    CALL pulse_e
    RETURN

start:
    MOVLW 0x60
    MOVWF OSCCON
    CLRF TRISB

    ; Init LCD
    CALL delay_5ms
    MOVLW 0x03
    MOVWF LATB
    CALL pulse_e
    MOVLW 0x02
    MOVWF LATB
    CALL pulse_e
    MOVLW 0x28
    CALL lcd_cmd
    MOVLW 0x0C
    CALL lcd_cmd

    ; Init UART
    BSF TRISC, 7
    BCF TRISC, 6
    MOVLW .25
    MOVWF SPBRG
    MOVLW 0x90
    MOVWF RCSTA

```

```

    MOVLW 0x24
    MOVWF TXSTA

prepare_new_msg:
    MOVLW 0x01
    CALL lcd_cmd
    BCF RCSTA, CREN
    MOVF RCREG, W
    MOVF RCREG, W
    BSF RCSTA, CREN
    LFSR FSR0, MSG_RAM_START

wait_char:
    BTFSS PIR1, RCIF
    BRA wait_char

    MOVF RCREG, W
    MOVWF POSTINC0
    XORLW 0x00
    BNZ wait_char

main_scroll:
    CLRF offset
scroll_step:
    MOVLW 0x80
    CALL lcd_cmd

    LFSR FSR0, MSG_RAM_START
    MOVF offset, W
    ADDWF FSR0L, 1

    MOVLW .16
    MOVWF char_count
display_window:
    BTFSC PIR1, RCIF
    BRA prepare_new_msg

    MOVF INDF0, W      ; Analyse sans déplacer le pointeur d'offset
    BZ restart_scroll

    MOVF POSTINC0, W   ; Avancement strict pour l'affichage unitaire
    CALL lcd_write_char
    DECFSZ char_count, 1
    BRA display_window

    CALL delay_25ms
    INCF offset, 1
    BRA scroll_step

restart_scroll:
    BRA main_scroll
END

```



## 5. Protocole de Saisie et Validation des Commandes

Le micrologiciel implémente un analyseur syntaxique rudimentaire et robuste basé sur la scrutation du registre de réception UART. Deux séquences de caractères spécifiques régissent le comportement de l'ensemble :

- **Séquence d'effacement (&&) :** Lorsque deux caractères ET commercial consécutifs sont détectés par le PIC Émetteur, la routine `clear_lcd_and_reset` réinitialise le pointeur d'écriture de la RAM (`MSG_BASE`), remet à zéro le compteur de caractères `ptr_count` et vide l'afficheur local.
- **Séquence de transmission (- -) :** La détection de deux tirets successifs clôt la phase de saisie. Le PIC Émetteur bascule immédiatement dans sa boucle d'émission `send_buffer`. Les octets stockés en RAM sont sérialisés vers le récepteur. La transmission s'achève par l'envoi d'un caractère de contrôle nul (`0x00`) agissant comme une rupture de boucle pour le PIC Récepteur.

## 6. Conclusion

La réalisation de ce projet sous environnement simulé a validé les choix algorithmiques et l'encapsulation des protocoles applicatifs. L'utilisation d'instructions non destructives pour les pointeurs de registres indirects (`INDF0`) a résolu les instabilités d'affichage défilant constatées lors des phases de tests préliminaires. Ce modèle constitue une base solide et fiable pour le développement ultérieur de cartes électroniques matérielles communicantes.